

International Cybersecurity and Digital Forensics Academy (ICDFA)**CIP-B103: Network Forensics Fundamentals****LAB 7: DNS Introduction and Traffic Analysis****Three-Week Delivery Block 3/3 | 1-2 August 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Lab Number	Lab 7
Lab Title	DNS Introduction and Traffic Analysis
Student Name	Fuseini Imoru Kantuogaa
Student ID	17291
Instructor Name	Dr. Sarah James
Delivery Block	3/3 of 3
Scheduled Dates	1-2 August 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	dig_dns.pcap or locally generated DNS captures; optional smtp.pcap correlation

1. Source Materials and Slide Alignment

- Domain Name System Forensics: domain-to-IP resolution, hierarchical DNS, resolver/TLD/authoritative roles, dig queries, capture with tshark, query/response fields and DNS within email traffic.
- The slide practical captures dig and browser DNS traffic, examines A records, local resolver configuration, TTL and authoritative information.
- The exercise builds the normal DNS baseline required before analyzing spoofing in Lab 8.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

You are documenting normal DNS behaviour for an incident-response baseline. You will identify the configured resolver, capture controlled dig queries, analyze transaction IDs, names, record types, response codes, answers and TTL values, and correlate DNS resolution with a subsequent web or email connection.

3. Learning Outcomes

- Explain recursive and iterative DNS resolution at a foundational level.
- Use dig to query A, AAAA, MX and NS records.
- Capture and analyze DNS queries and responses with tshark/Wireshark.
- Identify transaction ID, query name, record type, response code, answer and TTL.
- Correlate DNS responses with later IP connections.
- Recognize normal variations such as multiple answers, CNAMEs, cache responses and IPv6 queries.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux or Ubuntu VM	Generate and analyze DNS traffic.
DNS utility	dig from dnsutils	Perform controlled queries.
Capture tools	tshark and Wireshark	Capture DNS traffic.
Evidence	dig_dns.pcap or new PCAPNG	Analysis source.
Resolver config	/etc/resolv.conf, resolvectl	Identify configured DNS service.
Optional browser	Firefox/Chrome	Generate multiple DNS lookups.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab7/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab7
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```

sudo apt update
sudo apt install -y dnsutils tshark wireshark
cat /etc/resolv.conf | tee reports/resolv_conf.txt
resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
wget -O evidence/dig_dns.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-
lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap
sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt

```

Screenshot required: Configured resolver information, evidence details and hashes.



```

(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B103-Lab7/{evidence,working,exported,reports,screenshots,scripts}

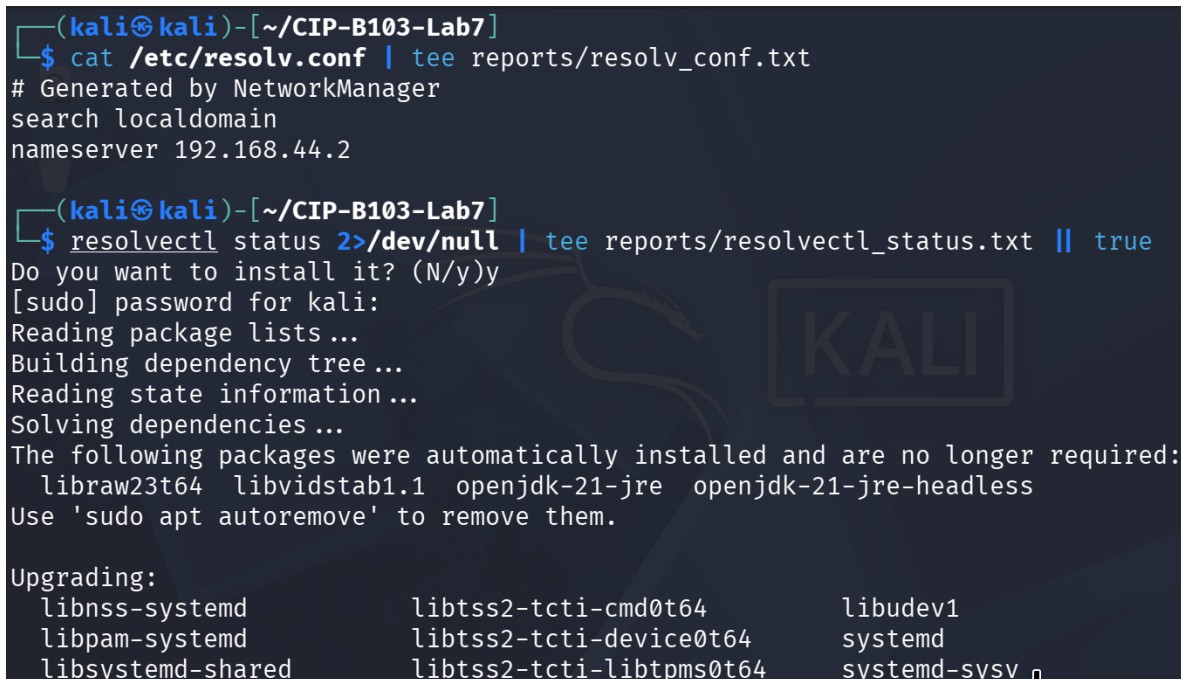
(kali㉿kali)-[~]
$ cd ~/CIP-B103-Lab7

(kali㉿kali)-[~/CIP-B103-Lab7]
$ pwd
/home/kali/CIP-B103-Lab7

(kali㉿kali)-[~/CIP-B103-Lab7]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported

```

Screenshot 1: Evidence folder structure created (mkdir, cd, pwd, find).



```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat /etc/resolv.conf | tee reports/resolv_conf.txt
# Generated by NetworkManager
search localdomain
nameserver 192.168.44.2

(kali㉿kali)-[~/CIP-B103-Lab7]
$ resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
Do you want to install it? (N/y)y
[sudo] password for kali:
Reading package lists...
Building dependency tree...
Reading state information...
Solving dependencies...
The following packages were automatically installed and are no longer required:
  libraw23t64 libvidstab1.1 openjdk-21-jre openjdk-21-jre-headless
Use 'sudo apt autoremove' to remove them.

Upgrading:
  libnss-systemd          libtss2-tcti-cmd0t64      libudev1
  libpam-systemd          libtss2-tcti-device0t64  systemd
  libsystemd-shared       libtss2-tcti-libtpms0t64 systemd-sysv

```

Screenshot 2: /etc/resolv.conf recorded and systemd-resolved installation started.

```

libtss2-mu-4.0.1-0t64      libtss2-tcti-swtpm0t64      tpm-udev
libtss2-sys1t64           libtss2-tctildr0t64         udev

Installing:
systemd-resolved

Installing dependencies:
libtss2-rc0t64  systemd-tpm

Suggested packages:
libnss-myhostname  libnss-resolve

Summary:
Upgrading: 21, Installing: 3, Removing: 0, Not Upgrading: 1717
Download size: 931 kB / 9,808 kB
Space needed: 3,121 kB / 23.4 GB available

Continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 libtss2-rc0t64 amd64 4.1.3-7 [31.1 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libtss2-mu-4.0.1-0t64 amd64 4.1.3-7 [77.5 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 libtss2-tcti-cmd0t64 amd64 4.1.3-7 [34.4 kB]

```

Screenshot 3: systemd-resolved package installation continued.

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ resolvectl status
Global
    Protocols: +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    resolv.conf mode: stub

Link 2 (eth0)
    Current Scopes: LLMNR/IPv4 LLMNR/IPv6
    Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    Default Route: no

Link 3 (docker0)
    Current Scopes: none
    Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    Default Route: no

```

Screenshot 4: resolvectl status showing the stub resolver and per-link protocol scopes.

```

(kali㉿kali)-[~]
$ cd ~/CIP-B103-Lab7

(kali㉿kali)-[~/CIP-B103-Lab7]
$ wget -O evidence/dig_dns.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
--2026-08-10 08:59:35-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1804656 (1.7M) [application/octet-stream]
Saving to: 'evidence/dig_dns.pcap'

evidence/dig_dns.pcap 100%[=====>] 1.72M 956KB/s in 1.8s

```

Screenshot 5: dig_dns.pcap downloaded via wget from the course GitHub repository.

```

2026-08-10 08:59:37 (956 KB/s) - 'evidence/dig_dns.pcap' saved [1804656/1804656]

(kali㉿kali)-[~/CIP-B103-Lab7]
$ ls -lh evidence/dig_dns.pcap
-rw-rw-r-- 1 kali kali 1.8M Aug 10 08:59 evidence/dig_dns.pcap

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  working/dig_dns_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab7]

```

Screenshot 6: Evidence file listed, copied to a working copy, and SHA-256 hashed (evidence and working copy identical).

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ find ~/CIP-B103-Lab7 -maxdepth 2 -type f -print
/home/kali/CIP-B103-Lab7/working/dig_dns_working.pcap
/home/kali/CIP-B103-Lab7/evidence/dig_dns.pcap
/home/kali/CIP-B103-Lab7/reports/resolvectl_status.txt
/home/kali/CIP-B103-Lab7/reports/resolv_conf.txt
/home/kali/CIP-B103-Lab7/reports/dns_capture_hashes.txt

(kali㉿kali)-[~/CIP-B103-Lab7]
$ █

```

Screenshot 7: Evidence folder contents confirmed via find.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab7- Fuseini Imoru Kantuogaa
Trainee name	Fuseini Imoru Kantuogaa
Date and time started	Mon 10 August 2026, approx. 08:59 EDT (Kali VM) — timestamp of the wget download of the reference evidence file
Evidence file name(s)	dig_dns.pcap (working copy: dig_dns_working.pcap); locally generated evidence/fresh_dig_dns.pcapng and evidence/browser_dns.pcapng
Source or generation method	dig_dns.pcap downloaded via wget from the frankwxu digital-forensics-lab GitHub repository (supplied reference evidence); fresh_dig_dns.pcapng and browser_dns.pcapng captured locally with tshark while running dig and browsing Firefox

Field	Student Entry
Original SHA-256	dig_dns.pcap: 9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d; fresh_dig_dns.pcapng: 70f60ab2c65fb1e10746f4da32af9895038a4fe5c9765ef72a08ebf7b4dc05aa; browser_dns.pcapng: b1a1d3a79210775d564539200312ee8e6c2f6173cd5fe7bce0bcc659c271de84
Working-copy SHA-256	dig_dns_working.pcap: 9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d — identical to original
Analysis workstation/VM	Kali Linux (user: kali), CIP-B103-Lab7 working directory; resolver 127.0.0.53 (systemd-resolved stub) forwarding to 192.168.44.2
Notes on any changes	cp --preserve=timestamps used for the working copy; SHA-256 of evidence and working copies matched exactly. tshark -i any produced a harmless 'promiscuous mode not supported' warning for the fresh capture but still recorded all 4 packets. The supplied dig_dns.pcap reference file was hashed but not directly analyzed with tshark in the available screenshots — analysis instead focused on the locally generated fresh_dig_dns.pcapng and browser_dns.pcapng.

8. Part A - Query DNS Records with dig

```
dig example.com A | tee reports/dig_example_A.txt
dig example.com AAAA | tee reports/dig_example_AAAA.txt
dig example.com MX | tee reports/dig_example_MX.txt
dig example.com NS | tee reports/dig_example_NS.txt
dig +short example.com A | tee reports/dig_example_short.txt
```

- Record the resolver shown in the SERVER line.
- Record status/response code, flags, question count, answer count and query time.
- Explain IN, A, AAAA, MX and NS.
- Record at least one TTL and explain that it is a cache lifetime, not a creation date.

```
(kali@kali)-[~/CIP-B103-Lab7]
$ dig example.com A | tee reports/dig_example_A.txt

; <<>> DiG 9.20.20-1-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 58570
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 5       IN      A      104.20.23.154
example.com.                 5       IN      A      172.66.147.243
```

Screenshot 8: dig example.com A executed and piped to reports/dig_example_A.txt.

```
;; Query time: 28 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Aug 10 09:03:30 EDT 2026
;; MSG SIZE rcvd: 72

(kali㉿kali)-[~/CIP-B103-Lab7]
$ dig example.com AAAA | tee reports/dig_example_AAAA.txt

; <<>> DiG 9.20.20-1-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 37317
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
```

Screenshot 9: A-record answer section and query time; dig example.com AAAA started.

```
;example.com.                IN      AAAA

;; ANSWER SECTION:
example.com.                5       IN      AAAA    2606:4700:10::ac42:93f3
example.com.                5       IN      AAAA    2606:4700:10::6814:179a

;; Query time: 39 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Aug 10 09:03:43 EDT 2026
;; MSG SIZE rcvd: 96

(kali㉿kali)-[~/CIP-B103-Lab7]
$ dig example.com MX | tee reports/dig_example_MX.txt

; <<>> DiG 9.20.20-1-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 49678
```

Screenshot 10: AAAA answer section; dig example.com MX started.

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      MX

;; ANSWER SECTION:
example.com.                5       IN      MX      0 .

;; Query time: 57 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Aug 10 09:03:59 EDT 2026
;; MSG SIZE rcvd: 55

(kali㉿kali)-[~/CIP-B103-Lab7]
$ dig example.com NS | tee reports/dig_example_NS.txt
```

Screenshot 11: MX answer section; dig example.com NS started.

```
(kali㉿kali)-[~/CIP-B103-Lab7]
$ dig example.com NS | tee reports/dig_example_NS.txt

; <<>> DiG 9.20.20-1-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; —>HEADER<— opcode: QUERY, status: NOERROR, id: 30531
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      NS

;; ANSWER SECTION:
example.com.                5       IN      NS      elliottns.cloudflare.com.
example.com.                5       IN      NS      herans.cloudflare.com.
```

Screenshot 12: NS answer section listing the authoritative Cloudflare nameservers.

```
;; Query time: 457 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Aug 10 09:04:12 EDT 2026
;; MSG SIZE rcvd: 95

(kali㉿kali)-[~/CIP-B103-Lab7]
$ dig +short example.com A | tee reports/dig_example_short.txt
172.66.147.243
104.20.23.154

(kali㉿kali)-[~/CIP-B103-Lab7]
$
```

Screenshot 13: NS query time/server; dig +short example.com A showing the bare IP answers.

9. Part B - Capture a Fresh DNS Query

```
IFACE=eth0
sudo tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng &
sleep 3
dig +noedns example.com A >/dev/null
wait
sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
```

Interface Note

On systems using a local stub resolver, packets may appear on lo and then on a physical interface. Use tshark -D to list interfaces and choose the one that shows the actual query.


```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP                192.168.44.128/24 fe80:
:fcdb:733e:9de1:56cb/64
docker0          DOWN              172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed
# by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If
# you're looking at
# /etc/resolv.conf and seeing this text, you have follo
# wed the symlink.
#
# This is a dynamic resolv.conf file for connecting loc
# al clients to the
# internal DNS stub resolver of systemd-resolved. This
# file lists all
# configured search domains.
#

```

Screenshot 14: Interface addresses and full resolv.conf (systemd-resolved stub) confirmed; IFACE set to any.

```

# Run "resolvectl status" to see details about the upli
nk DNS servers
# currently in use.
#
# Third party programs should typically not access this
# file directly, but only
# through the symlink at /etc/resolv.conf. To manage ma
n:resolv.conf(5) in a
# different way, replace this symlink by a static file
# or a different symlink.
#
# See man:systemd-resolved.service(8) for details about
# the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search localdomain

(kali㉿kali)-[~/CIP-B103-Lab7]
$ IFACE=any

(kali㉿kali)-[~/CIP-B103-Lab7]
$ echo "$IFACE"
any

```

Screenshot 15: Fresh DNS capture started on interface any (port 53, 30 s).

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo tshark -i "$IFACE" -f 'port 53' -a duration:30
-w /tmp/fresh_dig_dns.pcapng &
[2] 15885

(kali㉿kali)-[~/CIP-B103-Lab7]
$ Running as user "root" and group "root". This could
  be dangerous.
Capturing on 'any'
tshark: Promiscuous mode not supported on the "any" dev
ice.
4

[2]      done      sudo tshark -i "$IFACE" -f 'port 53'
-a duration:30 -w
(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo capinfos /tmp/fresh_dig_dns.pcapng
File name:                /tmp/fresh_dig_dns.pcapng
File type:                Wireshark/... - pcapng
File encapsulation:       Linux cooked-mode capture v1
File timestamp precision: nanoseconds (9)
Packet size limit:       file hdr: (not set)
Number of packets:       4
File size:                836 bytes
Data size:               356 bytes

```

Screenshot 16: capinfos on the fresh capture confirming 4 packets captured.

```

Latest packet time:      2026-08-10 09:27:23.065586834
Data byte rate:         12 kBps
Data bit rate:          96 kbps
Average packet size:    89.00 bytes
Average packet rate:    134 packets/s
SHA256:                 70f60ab2c65fb1e10746f4da32af989503
                        8a4fe5c9765ef72a08ebf7b4dc05aa
SHA1:                   f3258bd8da2487e33b795ef8e277dda74c
                        08255b
Strict time order:      True
Capture hardware:       Intel(R) Core(TM) i5-7300HQ CPU @
2.50GHz (with SSE4.2)
Capture oper-sys:       Linux 6.18.12+kali-amd64
Capture application:    Dumpcap (Wireshark) 4.6.6
Number of interfaces in file: 1
Interface #0 info:
                        Name = any
                        Encapsulation = Linux cooked-mode
capture v1 (25 - linux-sll)
                        Capture length = 262144
                        Time precision = nanoseconds (9)
                        Time ticks per second = 1000000000
                        Time resolution = 0x09
                        Filter string = port 53
                        Operating system = Linux 6.18.12+k

```

Screenshot 17: capinfos continued — SHA256/SHA1 and interface metadata.

```

ali-amd64
Number of stat entries = 1
Number of packets = 4

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo tshark -r /tmp/fresh_dig_dns.pcapng -Y dns
Running as user "root" and group "root". This could be
dangerous.
  1 0.000000000 127.0.0.1 → 127.0.0.53 DNS 73 St
standard query 0x15ad A example.com
  2 0.000524146 192.168.44.128 → 192.168.44.2 DNS 73
Standard query 0x0316 A example.com
  3 0.029427702 192.168.44.2 → 192.168.44.128 DNS 105
Standard query response 0x0316 A example.com A 172.66.
147.243 A 104.20.23.154
  4 0.029639633 127.0.0.53 → 127.0.0.1 DNS 105 S
tandard query response 0x15ad A example.com A 172.66.14
7.243 A 104.20.23.154

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo tshark -r /tmp/fresh_dig_dns.pcapng -Y 'dns.qr
y.name == "example.com"'
Running as user "root" and group "root". This could be
dangerous.
  1 0.000000000 127.0.0.1 → 127.0.0.53 DNS 73 St

```

Screenshot 18: capinfos concluded; tshark -Y dns listing all 4 query/response frames.

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ dig +noedns example.com A

; <<>> DiG 9.20.20-1-Debian <<>> +noedns example.com A
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 5549
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                  5       IN      A      172.66.147.243
example.com.                  5       IN      A      104.20.23.154

;; Query time: 28 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Aug 10 09:27:23 EDT 2026
;; MSG SIZE rcvd: 61

(kali㉿kali)-[~/CIP-B103-Lab7]
$ 

```

Screenshot 19: dig +noedns example.com A executed to generate the analyzed query.


```

1 0.000000000 127.0.0.1 → 127.0.0.53 DNS 73 Standard query 0x15ad A example.com
2 0.000524146 192.168.44.128 → 192.168.44.2 DNS 73 Standard query 0x0316 A example.com
3 0.029427702 192.168.44.2 → 192.168.44.128 DNS 105 Standard query response 0x0316 A example.com A 172.66.147.243 A 104.20.23.154
4 0.029639633 127.0.0.53 → 127.0.0.1 DNS 105 Standard query response 0x15ad A example.com A 172.66.147.243 A 104.20.23.154

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo cp --preserve=timestamps /tmp/fresh_dig_dns.pcapng evidence/fresh_dig_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo chown kali:kali evidence/fresh_dig_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
$ ls -lh evidence/fresh_dig_dns.pcapng
-rw-r--r-- 1 kali kali 836 Aug 10 09:27 evidence/fresh_dig_dns.pcapng

```

Screenshot 20: Frame listing repeated; capture copied and chown'd to evidence/.

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ capinfos evidence/fresh_dig_dns.pcapng
File name:          evidence/fresh_dig_dns.pcapng
File type:          Wireshark/... - pcapng
File encapsulation: Linux cooked-mode capture v1
File timestamp precision: nanoseconds (9)
Packet size limit:  file hdr: (not set)
Number of packets:  4
File size:          836 bytes
Data size:          356 bytes
Capture duration:   0.029639633 seconds
Earliest packet time: 2026-08-10 09:27:23.035947201
Latest packet time:  2026-08-10 09:27:23.065586834
Data byte rate:     12 kBps
Data bit rate:      96 kbps
Average packet size: 89.00 bytes
Average packet rate: 134 packets/s
SHA256:             70f60ab2c65fb1e10746f4da32af989503
8a4fe5c9765ef72a08ebf7b4dc05aa
SHA1:               f3258bd8da2487e33b795ef8e277dda74c
08255b
Strict time order:  True

```

Screenshot 21: evidence/fresh_dig_dns.pcapng listed; capinfos re-run on the evidence copy.

```

Strict time order:      True
Capture hardware:      Intel(R) Core(TM) i5-7300HQ CPU @
2.50GHz (with SSE4.2)
Capture oper-sys:      Linux 6.18.12+kali-amd64
Capture application:    Dumpcap (Wireshark) 4.6.6
Number of interfaces in file: 1
Interface #0 info:
                        Name = any
                        Encapsulation = Linux cooked-mode
capture v1 (25 - linux-sll)
                        Capture length = 262144
                        Time precision = nanoseconds (9)
                        Time ticks per second = 1000000000
                        Time resolution = 0x09
                        Filter string = port 53
                        Operating system = Linux 6.18.12+k
ali-amd64
                        Number of stat entries = 1
                        Number of packets = 4

```

```

(kali@kali)-[~/CIP-B103-Lab7]
$ sha256sum evidence/fresh_dig_dns.pcapng | tee repor

```

Screenshot 22: capinfos continued — timestamps, SHA256/SHA1 and interface metadata.

```

(kali@kali)-[~/CIP-B103-Lab7]
$ sha256sum evidence/fresh_dig_dns.pcapng | tee repor
ts/fresh_dns_sha256.txt
70f60ab2c65fb1e10746f4da32af9895038a4fe5c9765ef72a08ebf
7b4dc05aa evidence/fresh_dig_dns.pcapng

```

```

(kali@kali)-[~/CIP-B103-Lab7]
$ cat reports/fresh_dns_sha256.txt
70f60ab2c65fb1e10746f4da32af9895038a4fe5c9765ef72a08ebf
7b4dc05aa evidence/fresh_dig_dns.pcapng

```

```

(kali@kali)-[~/CIP-B103-Lab7]
$

```

Screenshot 23: evidence/fresh_dig_dns.pcapng SHA-256 hashed and confirmed via cat.

10. Part C - Extract DNS Query and Response Fields

```
PCAP=evidence/fresh_dig_dns.pcapng
```

```
# Query
```

```
tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv
```

```
# Response
```

```
tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
```

```
(kali㉿kali)-[~/CIP-B103-Lab7]
$ PCAP=evidence/fresh_dig_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
$ echo "$PCAP"
evidence/fresh_dig_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
$ tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv
1      2026-08-10T09:27:23.035947201-0400      127.0.0.1      44952      127.0.0.53      53      0x1
5ad     example.com      1
2      2026-08-10T09:27:23.036471347-0400      192.168.44.128  44072      192.168.44.2      53      0x0
316     example.com      1

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/dns_queries.tsv
1      2026-08-10T09:27:23.035947201-0400      127.0.0.1      44952      127.0.0.53      53      0x1
5ad     example.com      1
```

Screenshot 24: *dns.flags.response==0* field extraction showing both DNS query legs (stub and upstream).

```
2      2026-08-10T09:27:23.036471347-0400      192.168.44.128  44072      192.168.44.2      53      0x0
316     example.com      1

(kali㉿kali)-[~/CIP-B103-Lab7]
$ tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
3      2026-08-10T09:27:23.065374903-0400      192.168.44.2      53      192.168.44.128  44072      0x0
316     0      2      172.66.147.243,104.20.23.154      5,5
4      2026-08-10T09:27:23.065586834-0400      127.0.0.53      53      127.0.0.1      44952      0x1
5ad     0      2      172.66.147.243,104.20.23.154      5,5

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/dns_responses.tsv
3      2026-08-10T09:27:23.065374903-0400      192.168.44.2      53      192.168.44.128  44072      0x0
316     0      2      172.66.147.243,104.20.23.154      5,5
4      2026-08-10T09:27:23.065586834-0400      127.0.0.53      53      127.0.0.1      44952      0x1
5ad     0      2      172.66.147.243,104.20.23.154      5,5

(kali㉿kali)-[~/CIP-B103-Lab7]
```

Screenshot 25: *dns.flags.response==1* field extraction showing both DNS response legs with answers and TTLs.

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ ls -lh reports/dns_queries.tsv reports/dns_responses.tsv
-rw-rw-r-- 1 kali kali 183 Aug 10 09:40 reports/dns_queries.tsv
-rw-rw-r-- 1 kali kali 231 Aug 10 09:41 reports/dns_responses.tsv

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/dns_queries.tsv
1      2026-08-10T09:27:23.035947201-0400      127.0.0.1      44952      127.0.0.53      53      0x1
5ad    example.com      1
2      2026-08-10T09:27:23.036471347-0400      192.168.44.128 44072      192.168.44.2      53      0x0
316    example.com      1

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/dns_responses.tsv
3      2026-08-10T09:27:23.065374903-0400      192.168.44.2      53      192.168.44.128 44072      0x0
316    0      2      172.66.147.243,104.20.23.154      5,5
4      2026-08-10T09:27:23.065586834-0400      127.0.0.53      53      127.0.0.1      44952      0x1
5ad    0      2      172.66.147.243,104.20.23.154      5,5

```

Screenshot 26: reports/dns_queries.tsv and reports/dns_responses.tsv confirmed via ls and cat.

11. Part D - Match Queries to Responses

Transaction ID	Query Name	Type	Client/Resolver	Response Code	Answer(s)	TTL	Time Delta
0x15ad	example.com	A	127.0.0.1:44952 → 127.0.0.53:53 (stub resolver)	NOERR (0)	172.66.147.243, 104.20.23.154	5 s (both answers)	29.639 ms (frame 1 → frame 4, 0.00000000 → 0.029639633 s)
0x0316	example.com	A	192.168.44.128:44072 → 192.168.44.2:53 (upstream/gateway resolver)	NOERR (0)	172.66.147.243, 104.20.23.154	5 s (both answers)	28.903 ms (frame 2 → frame 3, 0.000524146 → 0.029427702 s)
Both IDs confirmed distinct per leg (0x15ad stub, 0x0316 upstream)	example.com (dig +noedns example.com A)	A	Two-hop resolution: client → local stub (127.0.0.53) → upstream resolver (192.168.44.2)	NOERR for both legs	Identical answer set on both legs: 172.66.147.243, 104.20.23.154	5 s uniformly	Overall round trip ≈ 29.6 ms from initial stub query to final stub response

- Match query and response using transaction ID plus the endpoint tuple.
- Confirm that the response source is the queried resolver.
- Calculate response time from frame timestamps.

- Document CNAME chains and multiple A/AAAA answers where present.

12. Part E - Analyze DNS Generated by a Browser

A browser may request several domains for the page, images, fonts, telemetry or cached services. Capture a visit to a non-sensitive instructor-approved site and inventory all query names.

```
sudo tshark -i IFACE -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng &
sleep 3
# Open the approved site in a private browser window, then wait.
wait
```

```
tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e
dns.qry.type \
| sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
```



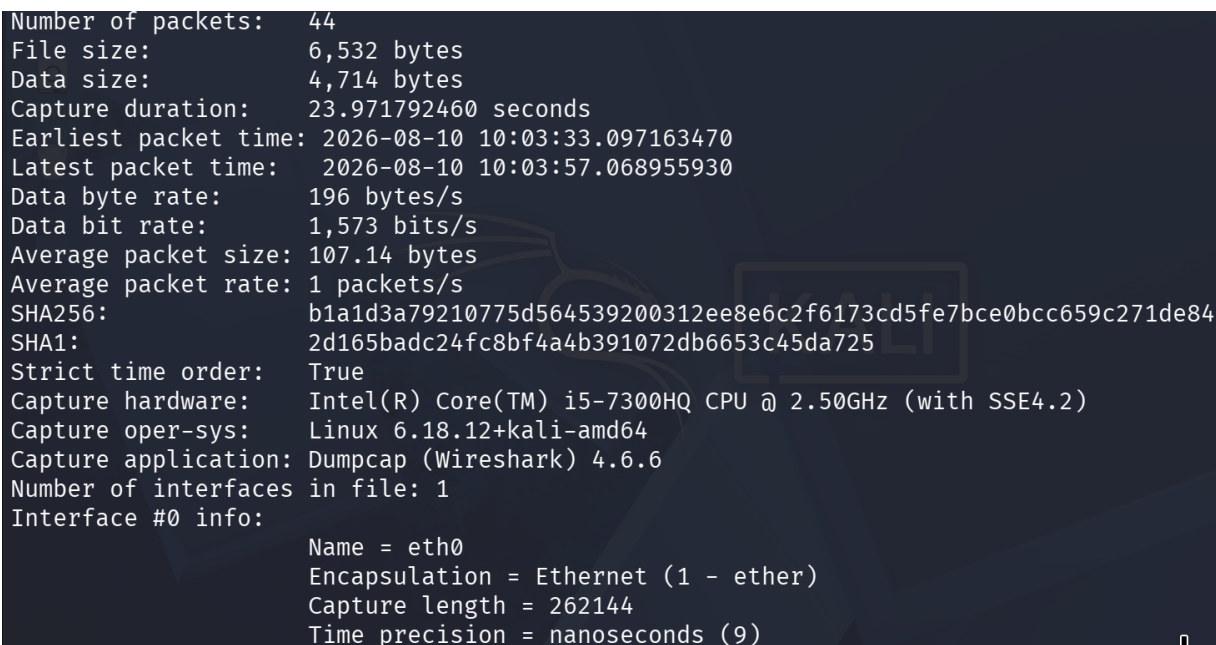
```
(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo -v

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo tshark -i eth0 -p -f 'port 53' -a duration:60 -w /tmp/browser_dns.pcapng &
[5] 33037

(kali㉿kali)-[~/CIP-B103-Lab7]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
44

[5] done sudo tshark -i eth0 -p -f 'port 53' -a duration:60 -w /tmp/browser_dns.pcapng
(kali㉿kali)-[~/CIP-B103-Lab7]
$ sudo capinfos /tmp/browser_dns.pcapng
File name: /tmp/browser_dns.pcapng
File type: Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit: file hdr: (not set)
Number of packets: 44
```

Screenshot 27: Browser DNS capture started on eth0 (port 53, 60 s).



```
Number of packets: 44
File size: 6,532 bytes
Data size: 4,714 bytes
Capture duration: 23.971792460 seconds
Earliest packet time: 2026-08-10 10:03:33.097163470
Latest packet time: 2026-08-10 10:03:57.068955930
Data byte rate: 196 bytes/s
Data bit rate: 1,573 bits/s
Average packet size: 107.14 bytes
Average packet rate: 1 packets/s
SHA256: b1a1d3a79210775d564539200312ee8e6c2f6173cd5fe7bce0bcc659c271de84
SHA1: 2d165badc24fc8bf4a4b391072db6653c45da725
Strict time order: True
Capture hardware: Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz (with SSE4.2)
Capture oper-sys: Linux 6.18.12+kali-amd64
Capture application: Dumpcap (Wireshark) 4.6.6
Number of interfaces in file: 1
Interface #0 info:
    Name = eth0
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = nanoseconds (9)
```

Screenshot 28: capinfos on the raw browser capture confirming 44 packets.

```

└─$ sudo cp --preserve=timestamps /tmp/browser_dns.pcapng evidence/browser_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
└─$ sudo chown kali:kali evidence/browser_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
└─$ ls -lh evidence/browser_dns.pcapng
-rw----- 1 kali kali 6.4K Aug 10 10:03 evidence/browser_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
└─$ capinfos evidence/browser_dns.pcapng
File name:          evidence/browser_dns.pcapng
File type:          Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit:  file hdr: (not set)
Number of packets:  44
File size:          6,532 bytes
Data size:          4,714 bytes
Capture duration:   23.971792460 seconds
Earliest packet time: 2026-08-10 10:03:33.097163470
Latest packet time:  2026-08-10 10:03:57.068955930

```

Screenshot 29: Browser capture copied to evidence/ and chown'd; capinfos re-run on the evidence copy.

```

Average packet size: 107.14 bytes
Average packet rate: 1 packets/s
SHA256:              b1a1d3a79210775d564539200312ee8e6c2f6173cd5fe7bce0bcc659c271de84
SHA1:                2d165badc24fc8bf4a4b391072db6653c45da725
Strict time order:   True
Capture hardware:    Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz (with SSE4.2)
Capture oper-sys:    Linux 6.18.12+kali-amd64
Capture application: Dumpcap (Wireshark) 4.6.6
Number of interfaces in file: 1
Interface #0 info:
    Name = eth0
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = nanoseconds (9)
    Time ticks per second = 1000000000
    Time resolution = 0x09
    Filter string = port 53
    Operating system = Linux 6.18.12+kali-amd64
    Number of stat entries = 1
    Number of packets = 44

(kali㉿kali)-[~/CIP-B103-Lab7]

```

Screenshot 30: capinfos continued — SHA256/SHA1 and interface metadata for the evidence copy.


```
(kali㉿kali)-[~/CIP-B103-Lab7]
$ tshark -r evidence/browser_dns.pcapng \
-Y 'dns.flags.response==0' \
-T fields \
-e dns.qry.name \
-e dns.qry.type \
| sort | uniq -c | sort -nr \
| tee reports/browser_dns_inventory.txt
1 push.services.mozilla.com 28
1 push.services.mozilla.com 1
1 mozilla.map.fastly.net 65
1 ipv4only.arpa 28
1 ipv4only.arpa 1
1 firefox.settings.services.mozilla.com 28
1 firefox.settings.services.mozilla.com 1
1 firefox-settings-attachments.cdn.mozilla.net 65
1 firefox-settings-attachments.cdn.mozilla.net 28
1 firefox-settings-attachments.cdn.mozilla.net 1
1 example.org 28
1 example.org 1
1 example.com 65
1 example.com 28
```

Screenshot 31: Browser DNS query inventory (sorted, deduplicated) — Mozilla telemetry/CDN and example domains.

```
1 detectportal.firefox.com 1
1 content-signature-2.cdn.mozilla.net 28
1 content-signature-2.cdn.mozilla.net 1
1 cloudflare-dns.com 65
1 ads.mozilla.org 28
1 ads.mozilla.org 1

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/browser_dns_inventory.txt
1 push.services.mozilla.com 28
1 push.services.mozilla.com 1
1 mozilla.map.fastly.net 65
1 ipv4only.arpa 28
1 ipv4only.arpa 1
1 firefox.settings.services.mozilla.com 28
1 firefox.settings.services.mozilla.com 1
1 firefox-settings-attachments.cdn.mozilla.net 65
1 firefox-settings-attachments.cdn.mozilla.net 28
1 firefox-settings-attachments.cdn.mozilla.net 1
1 example.org 28
1 example.org 1
1 example.com 65
```

Screenshot 32: Query inventory continued, including the DoH probe to cloudflare-dns.com.

```
1 example.com 65
1 example.com 28
1 example.com 1
1 detectportal.firefox.com 28
1 detectportal.firefox.com 1
1 content-signature-2.cdn.mozilla.net 28
1 content-signature-2.cdn.mozilla.net 1
1 cloudflare-dns.com 65
1 ads.mozilla.org 28
1 ads.mozilla.org 1

(kali㉿kali)-[~/CIP-B103-Lab7]
$ column -t reports/browser_dns_inventory.txt
1 push.services.mozilla.com 28
1 push.services.mozilla.com 1
1 mozilla.map.fastly.net 65
1 ipv4only.arpa 28
1 ipv4only.arpa 1
1 firefox.settings.services.mozilla.com 28
1 firefox.settings.services.mozilla.com 1
1 firefox-settings-attachments.cdn.mozilla.net 65
1 firefox-settings-attachments.cdn.mozilla.net 28
```

Screenshot 33: reports/browser_dns_inventory.txt contents confirmed via cat.

```

1  firefox-settings-attachments.cdn.mozilla.net 1
1  example.org 28
1  example.org 1
1  example.com 65
1  example.com 28
1  example.com 1
1  detectportal.firefox.com 28
1  detectportal.firefox.com 1
1  content-signature-2.cdn.mozilla.net 28
1  content-signature-2.cdn.mozilla.net 1
1  cloudflare-dns.com 65
1  ads.mozilla.org 28
1  ads.mozilla.org 1

(kali㉿kali)-[~/CIP-B103-Lab7]
$ tshark -r evidence/browser_dns.pcapng \
-Y 'dns.flags.response==0' \
-T fields -e dns.qry.name \
| sort -u \
| tee reports/browser_dns_unique_domains.txt
ads.mozilla.org

```

Screenshot 34: Inventory re-confirmed with column -t; unique-domain extraction started.

```

cloudflare-dns.com
content-signature-2.cdn.mozilla.net
detectportal.firefox.com
example.com
example.org
firefox-settings-attachments.cdn.mozilla.net
firefox.settings.services.mozilla.com
ipv4only.arpa
mozilla.map.fastly.net
push.services.mozilla.com

(kali㉿kali)-[~/CIP-B103-Lab7]
$ sha256sum evidence/browser_dns.pcapng | tee reports/browser_dns_sha256.txt
b1a1d3a79210775d564539200312ee8e6c2f6173cd5fe7bce0bcc659c271de84  evidence/browser_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/browser_dns_sha256.txt
b1a1d3a79210775d564539200312ee8e6c2f6173cd5fe7bce0bcc659c271de84  evidence/browser_dns.pcapng

(kali㉿kali)-[~/CIP-B103-Lab7]

```

Screenshot 35: Unique domain list completed; evidence/browser_dns.pcapng SHA-256 hashed.

```

$ tree
.
├── evidence
│   ├── browser_dns.pcapng
│   ├── dig_dns.pcap
│   └── fresh_dig_dns.pcapng
├── exported
├── reports
│   ├── browser_dns_inventory.txt
│   ├── browser_dns_sha256.txt
│   ├── browser_dns_unique_domains.txt
│   ├── dig_example_AAAA.txt
│   ├── dig_example_A.txt
│   ├── dig_example_MX.txt
│   ├── dig_example_NS.txt
│   ├── dig_example_short.txt
│   ├── dns_capture_hashes.txt
│   ├── dns_queries.tsv
│   ├── dns_responses.tsv
│   ├── fresh_dns_sha256.txt
│   ├── resolv_conf.txt
│   └── resolvectl_status.txt

```

Screenshot 36: Full evidence and reports folder tree after Part E.



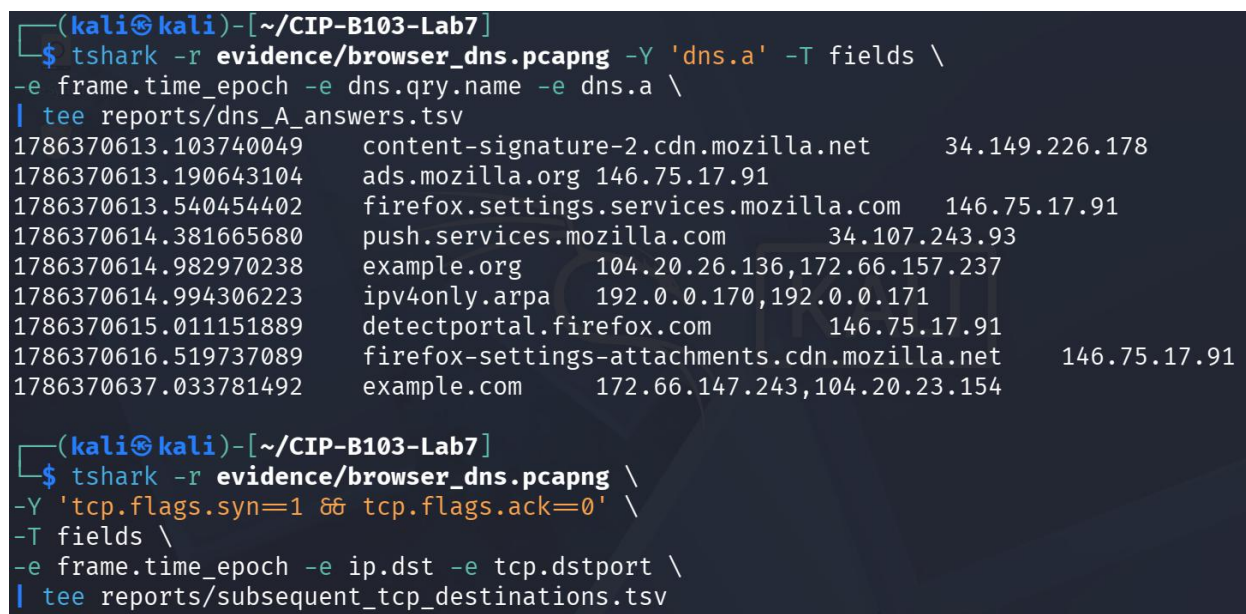
Screenshot 37: Folder tree concluded — working copy and total file count.

13. Part F - Correlate DNS with Subsequent Connections

```
# Extract DNS A answers
tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers.tsv

# Extract later TCP destinations
tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations.tsv
```

Select one DNS answer and show whether the client later initiates a connection to that IP. Explain any mismatch caused by multiple answers, proxies, CDNs, IPv6, caching or capture duration.



Screenshot 38: DNS A-answer extraction (dns.a) from browser_dns.pcapng, and the (empty) subsequent-TCP-destination query started.


```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/dns_A_answers.tsv
1786370613.103740049 content-signature-2.cdn.mozilla.net 34.149.226.178
1786370613.190643104 ads.mozilla.org 146.75.17.91
1786370613.540454402 firefox.settings.services.mozilla.com 146.75.17.91
1786370614.381665680 push.services.mozilla.com 34.107.243.93
1786370614.982970238 example.org 104.20.26.136,172.66.157.237
1786370614.994306223 ipv4only.arpa 192.0.0.170,192.0.0.171
1786370615.011151889 detectportal.firefox.com 146.75.17.91
1786370616.519737089 firefox-settings-attachments.cdn.mozilla.net 146.75.17.91
1786370637.033781492 example.com 172.66.147.243,104.20.23.154

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/subsequent_tcp_destinations.tsv

(kali㉿kali)-[~/CIP-B103-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y tcp -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
-e tcp.srcport -e tcp.dstport -e tcp.flags

(kali㉿kali)-[~/CIP-B103-Lab7]
$

```

Screenshot 39: *dns_A_answers.tsv* contents confirmed; *subsequent_tcp_destinations.tsv* confirmed empty (capture was DNS-only, port 53).

14. Part G - DNS Analysis Within SMTP Evidence

If *smtp.pcap* from Lab 4 is available, filter dns and identify the query used to locate a mail server. Record the queried name, record type, resolver, answer and TTL.

Optional, adjust path to your Lab 4 capture

```

tshark -r ../CIP-B103-Lab4/working/smtp_working.pcap -Y 'dns' -T fields \
-e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e dns.resp.ttl \
| tee reports/smtp_dns_correlation.tsv

```

```

(kali㉿kali)-[~/CIP-B103-Lab7]
$ ls -lh ../CIP-B103-Lab4/working/smtp_working.pcap
-rw-rw-r-- 1 kali kali 28K Aug  1 08:42 ../CIP-B103-Lab4/working/smtp_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab7]
$ tshark -r ../CIP-B103-Lab4/working/smtp_working.pcap \
-Y 'dns' -T fields \
-e frame.number -e frame.time \
-e dns.qry.name -e dns.qry.type \
-e dns.a -e dns.resp.name -e dns.resp.ttl \
| tee reports/smtp_dns_correlation.tsv
1      2009-10-05T02:06:07.492060000-0400      mail.patriots.in      1
2      2009-10-05T02:06:07.526085000-0400      mail.patriots.in      1      74.53.140.153      mai
l.patriots.in,patriots.in,patriots.in,patriots.in      10827,10828,82828,82828

(kali㉿kali)-[~/CIP-B103-Lab7]
$ cat reports/smtp_dns_correlation.tsv
1      2009-10-05T02:06:07.492060000-0400      mail.patriots.in      1
2      2009-10-05T02:06:07.526085000-0400      mail.patriots.in      1      74.53.140.153      mai
l.patriots.in,patriots.in,patriots.in,patriots.in      10827,10828,82828,82828

```

Screenshot 40: DNS records extracted from the Lab 4 *smtp_working.pcap*, showing the query used to locate *mail.patriots.in*.

15. Required Findings Worksheet

Field	Finding
Configured resolver IP	127.0.0.53 (systemd-resolved local stub), forwarding upstream to 192.168.44.2
Client source port	44952 (client → stub leg); 44072 (stub → upstream leg)
Resolver destination port	53 (UDP) on both legs
Transaction ID	0x15ad (client–stub leg); 0x0316 (stub–upstream leg) — the stub resolver assigns a new ID for the upstream query
Query name and type	example.com, type A
Response code	NOERROR (0) on both legs
Answer IP(s)	172.66.147.243 and 104.20.23.154 (two A records, Cloudflare-hosted)
TTL	5 seconds for both answers — a short cache lifetime, not a record creation date
Query-response time delta	≈29.6 ms end-to-end (client query at 0.000000000 s to final stub response at 0.029639633 s); the upstream leg alone took ≈28.9 ms
Subsequent connection correlation	Not observed — reports/subsequent_tcp_destinations.tsv is empty. This is a methodological limitation, not a real absence of a connection: the browser_dns.pcapng capture filter was restricted to ‘port 53’, so no TCP traffic (including any HTTP/HTTPS connection to the resolved IPs) could ever have been recorded in that file.
Normal variations observed	Multiple A answers per query (2 IPs for example.com); a two-hop local-stub-to-upstream-resolver pattern with a different transaction ID per hop; a wide browser DNS inventory including telemetry/CDN domains (Mozilla push/settings services, content-signature CDN, DoH probe to cloudflare-dns.com, ipv4only.arpa NAT64 check) alongside the requested site; consistently short (5 s) TTLs typical of a CDN-fronted domain.

Required Screenshots Checklist

☐ Resolver configuration.

- ☐ dig A/AAAA/MX/NS outputs.
- ☐ Fresh DNS capture hash.
- ☐ DNS query fields.
- ☐ Matching DNS response fields.
- ☐ Transaction ID correlation.
- ☐ Answer and TTL.
- ☐ Browser query inventory.
- ☐ DNS answer to TCP destination correlation.
- ☐ Optional SMTP DNS evidence.

Student Report Template

1. Executive summary.
2. Resolver and environment.
3. Evidence integrity.
4. dig results.
5. Packet-level query/response analysis.
6. Transaction matching and timing.
7. Browser DNS inventory.
8. Connection correlation.
9. Limitations and normal variations.
10. Appendix.

Submission Requirements

- One PDF report named CIP-B103-Lab7_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Preparation and resolver identification	10	Correct resolver and evidence hashes.
dig record analysis	15	A, AAAA, MX, NS and TTL interpretation.
Packet-field extraction	25	Correct IDs, names, types, endpoints and response codes.

Assessment Area	Marks	What the Instructor Will Check
Query-response correlation	20	Accurate matching and timing.
Browser/connection correlation	20	Inventory and defensible IP connection linkage.
Report quality	10	Clear screenshots, tables and conclusions.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Standard transport	Usually UDP destination port 53; TCP may be used for large responses or zone transfer.
Query flag	<code>dns.flags.response==0.</code>
Response flag	<code>dns.flags.response==1.</code>
A record	IPv4 address.
AAAA record	IPv6 address.
TTL	How long a resolver may cache the record.

Command Reference Summary

Command	Purpose
<code>dig NAME TYPE</code>	Query a DNS record.
<code>dns.qry.name</code>	Queried domain name.
<code>dns.qry.type</code>	Requested record type.
<code>dns.id</code>	DNS transaction identifier.
<code>dns.flags.rcode</code>	Response code.
<code>dns.resp.ttl</code>	Answer TTL.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.